

Support-Unrestricted Cyclic Braid Kernels

Shared Garside prefixes, verifiable compression,
strict target-radius separations, and
quasi-polynomial recognition classes

Prepared for Vladimir Reshetnikov's ProveIt program

AI-assisted mathematical research draft

7 October 2026

Abstract

We develop an exact, certificate-producing preprocessing layer for the ProveIt unknot recognizer. It removes the two-generator support restriction of the existing local braid compressor and shares one table of Garside prefix states over every cyclic cut. A second trie shares arithmetic among finite target words. For a fixed target radius, the resulting optimizer is polynomial-time and returns an optimal set of disjoint substitutions for its explicit one-pass rule set. We give bit-complexity and certificate-size bounds, including a sharper bound in terms of the maximum canonical prefix height.

An explicit infinite family of four-strand unknots separates target radii one and two: the radius-one cyclic kernel leaves all $4m + 1$ letters, whereas radius two leaves exactly three. The proof uses an induced lattice path and a signed-crossing obstruction to single-letter replacements across the remaining generator. A different family consists of geodesic three-braid representatives of the unknot, proving that unrestricted braid equality shortening alone cannot replace closure-aware operations.

An uncapped composition with exact Khovanov homology has quasi-polynomial complexity on constructively recognizable small-kernel families, including conjugated flat multi-letter inflations. We do not prove a small-kernel bound for arbitrary unknots. The accompanying standard-library Python implementation includes an independent local-move verifier, a cheaper whole-word normal-form shortcut, 47 passing tests, exhaustive independent checks, raw paired measurements, and integration instructions. Measured speedups concern shared versus duplicated prefix algebra, not the complete upstream recognizer.

Status. Mathematical proofs are supplied below. Implementation validation is by independent replay and finite tests, not a Lean formalization. No general quasi-polynomial unknot-recognition theorem or end-to-end upstream speedup is claimed.

Contents

1	Research context and precise scope	2
2	Braid words, kernels, and equality scopes	3
3	An exact classical Garside engine	4
4	Shared prefixes and an exact cyclic optimizer	6
5	Bit complexity, storage, and certificate size	8
6	Independent replay and the trust boundary	9
7	An infinite strict separation of target radii	10
8	Further obstructions and the role of cyclic closure	12
9	A constructive quasi-polynomial recognition class	14
10	Implementation validation and measurements	16
11	Integration into the maintained recognizer	18
12	Further research questions	18
13	Conclusion	20
A	Reproduction and artifact map	20
B	Audit questions for a reviewer	21

1 Research context and precise scope

1.1 The current repository bottleneck

The inspected ProveIt tree already contains exact Khovanov-based recognition, structural certificates, homological windows, twist compression, component sharing, and optional rank-two braid preprocessing [4, 5]. Its rank-two stage is substantially more than a heuristic cancellation pass: it optimizes disjoint replacements of subwords supported on at most two Artin generator indices, checks exact local braid equality, and independently replays a certificate. Its near-linear bound exploits faithful low-rank states and bounded overlap of local corridors. The present note does not rediscover or replace that result.

The same maintained analysis exhibits four-strand unknot words on which no strictly shortening rank-two substitution can start. Increasing the number of identical passes does not solve that obstruction. This suggests two separate changes: remove the support restriction, and permit a recorded cyclic cut before searching. The first uses classical Garside normal forms; the second preserves a closed braid rather than the original braid element. Keeping those two equality scopes distinct is essential.

The broader quasi-polynomial goal is not settled by such preprocessing. Lackenby’s July 2026 preprint develops hierarchical constructions but its Section 9 explicitly does not provide enough algorithmic precision for a running-time estimate, and leaves the required hierarchy-parameter bounds unproved there [3]. We neither use an unspecified hierarchy operation as an oracle nor infer a general complexity theorem from an announcement. Our route below is an independent, parameterized improvement to the executable braid/Khovanov pipeline.

1.2 What is classical, extended, and proved here

Classical inputs are the Artin presentation of braid groups, the simple-braid permutation model, uniqueness of the left-greedy Garside normal form, and Khovanov detection of the unknot [1, 2]. The prefix-table dynamic program is an extension of the repository’s existing rank-two recurrence, not an independently new general dynamic-programming principle.

The contributions developed and proved in this note are the shared *doubled-prefix and inverse-target-trie* construction, its explicit height-sensitive cost accounting and replay format, the unrestricted cyclic kernel analysis, the infinite radius-one/radius-two separation, and the constructive recognition classes obtained by composing these results. We also give a geodesic unknot obstruction and a practical candidate portfolio. These are new contributions to this package relative to the inspected maintained sources. A comprehensive priority search over all braid-algorithm literature has not been performed; global historical novelty is not asserted.

The suggested `openai/math` repository was inspected, including its formalized positive-braid restriction lemma [7]. Its separation of a derivation from a support restriction is methodologically relevant. No new theorem from that repository is needed as an unverified premise here, and no claim is made that our Python implementation has been checked by its Lean library.

1.3 Claim ledger

Claim	Status and scope
Exact kernel optimization	Proved for disjoint one-pass substitutions from a fixed finite dictionary, with optional cyclic rotation.
Polynomial preprocessing	Proved for each fixed radius, with explicit dependence on strand count and prefix height.
Strict radius separation	Proved on an infinite family of actual four-strand unknots.
Quasi-polynomial recognition	Proved for the stated small-kernel families, using an uncapped exact fallback.
Implementation soundness	Independently replayable local certificates; 47 passing tests. No formal verification of Python semantics.
Performance	Measured compressor-level costs with raw data. No complete upstream benchmark was run.
General quasi-polynomial bound	Not established; no universal structural hypothesis is silently assumed.

2 Braid words, kernels, and equality scopes

Let

$$B_b = \langle \sigma_1, \dots, \sigma_{b-1} \mid \sigma_i \sigma_{i+1} \sigma_i = \sigma_{i+1} \sigma_i \sigma_{i+1}, \quad \sigma_i \sigma_j = \sigma_j \sigma_i \ (|i - j| \geq 2) \rangle.$$

A word $w = a_1 \cdots a_n$ is encoded by signed integers. Its literal length is n , not its minimum Artin length. An empty word on one strand closes to the unknot; an empty word on several strands is not a knot. The compressor itself is defined for braids whether or not their closures are connected.

For a finite dictionary D of literal target words, a permitted plan consists of disjoint intervals $[i, j)$ in the original word and replacements

$$w[i : j] = v \quad \text{in } B_b, \quad v \in D, \quad |v| < j - i.$$

Unreplaced letters are retained. The intervals are not interpreted in a word that has already been edited. This makes a one-pass plan unambiguous and prevents a hidden saturation loop from entering the complexity claim.

Definition 2.1 (Linear and cyclic kernels). Let $\kappa_D^{\text{lin}}(w)$ be the minimum output length over such plans. For $n > 0$, put

$$\text{rot}_s(w) = w[s : n]w[0 : s], \quad \kappa_D^{\text{cyc}}(w) = \min_{0 \leq s < n} \kappa_D^{\text{lin}}(\text{rot}_s(w)).$$

For the empty word the only cut is zero. The radius- ρ dictionary is

$$D_\rho = \{v : |v| \leq \rho\}, \quad M_\rho = |D_\rho| = \sum_{d=0}^{\rho} (2b - 2)^d.$$

We abbreviate its two optimum lengths by κ_ρ^{lin} and κ_ρ^{cyc} . Literal targets representing the same braid may both remain in the dictionary; the dynamic program accounts for their actual lengths.

A cyclic cut is conjugacy, not equality: if $w = uv$, then $vu = u^{-1}wu$. After the cut, every replacement is braid equality in its unchanged surrounding context. Therefore a cyclic certificate proves that the output closure is isotopic to the original closure. It does not assert $w = \text{rot}_s(w)$ in B_b .

The optimum here is not a minimum over all braid presentations, all sequences of length-preserving relations, or all Markov moves. It can remain large even when the original braid has a much shorter equal word. Section 7 gives an explicit infinite example. The separate portfolio stage can exploit such an equality without mislabeling it as a radius-one plan.

Lemma 2.2 (A safe stopping bound). *Let $e(w)$ be the signed exponent sum, and let $c(\pi(w))$ be the number of cycles of the induced permutation on b strands. Every word equal to a cyclic rotation of w has length at least*

$$L(w) = \max\{|e(w)|, b - c(\pi(w))\}.$$

Consequently a search attaining $L(w)$ may stop without checking the remaining cuts. If $L(w) = n$, the unchanged word is already optimal for every dictionary.

Proof. Each signed Artin letter contributes ± 1 to exponent sum. Its permutation is one transposition, which changes the number of cycles by exactly one. Starting from the identity permutation requires at least $b - c(\pi(w))$ transpositions. Exponent sum and permutation cycle type are conjugacy invariants. These bounds apply to every permitted output. $\square$

For a knot closure, $c(\pi(w)) = 1$, hence $b \leq n + 1$. After compression to k letters without changing strand count, the same argument gives $b \leq k + 1$. This removes a potentially hidden strand parameter from the recognition corollary. Polynomial dependence on a numerically huge strand count is not claimed to be polynomial in a succinct binary encoding of arbitrary many-component empty braids.

3 An exact classical Garside engine

3.1 Simple braids and the local transfer

Write $\Delta = \Delta_b$ for the positive half-twist, and $B = \binom{b}{2}$ for its Artin length. A simple positive braid crosses each pair of strands at most once and is uniquely determined by its permutation. We use the convention that $p[i]$ is the bottom position of top strand i , with indices $0, \dots, b - 1$. This is the inverse of conventions that record the top label seen at a bottom position.

In this convention the starting atoms of a simple braid p are its descents, and the finishing atoms are the descents of p^{-1} . The classical left-greedy normal form is

$$g = \Delta^z s_1 \cdots s_r, \tag{1}$$

where $z \in \mathbb{Z}$, each s_i is a nonidentity simple braid different from Δ , and every starting atom of s_{i+1} is a finishing atom of s_i . The existence and uniqueness of this form are classical [1]. We represent it by the integer z and the complete permutation arrays, not by a quotient or a probabilistic fingerprint.

Lemma 3.1 (Checkable transfer). *For adjacent simple factors A, C , an atom indexed by $i + 1$ can be moved from C to A whenever*

$$C[i] > C[i + 1], \quad A^{-1}[i] < A^{-1}[i + 1].$$

The new left factor swaps the values $i, i + 1$ in A , and the new right factor swaps positions $i, i + 1$ in C . Both remain simple and their product is unchanged. The right factor is removed if it becomes the identity.

Proof. The first inequality says that $C = \sigma_{i+1}C'$ with $\ell(C') = \ell(C) - 1$. The second says that $A\sigma_{i+1}$ is simple, with length $\ell(A) + 1$. Uniqueness of the simple braid for a permutation identifies the two updated arrays with these two factors. Thus $AC = (A\sigma_{i+1})C'$. The array operations are exactly right composition and left removal in the stated convention. $\square$

A positive input letter is appended as a simple factor. For a negative letter, use

$$\sigma_i^{-1} = \Delta^{-1}c_i, \quad c_i = \Delta\sigma_i^{-1}, \quad S\Delta^{-1} = \Delta^{-1}\tau(S),$$

where τ reverses the generator indices. On a permutation array,

$$\tau(p)[i] = b - 1 - p[b - 1 - i].$$

We decrease z by one, apply τ to all existing factors, and append c_i if it is nonidentity. Transfers are then performed until every adjacent pair is left weighted. Any half-twist factors are necessarily initial: a normal pair with a right factor Δ requires every atom to finish the left factor, so that factor must itself be Δ . Extracting the initial half-twists completes (1).

3.2 An elementary transfer potential

Theorem 3.2 (Amortized normalization cost). *Normalizing a signed m -letter word by the described atom-transfer algorithm uses at most*

$$B m(m-1)/2$$

transfers, at most m simultaneous simple factors, and $O(m^2b^3)$ word operations. Right-multiplying one already normal state of r factors by an atom costs $O((r+1)^2b^3)$ word operations. Integers have $O(\log(m+b+2))$ bits during whole-word normalization.

Proof. For a temporary list of factors, indexed from zero, define

$$\Phi(s_0, \dots, s_{r-1}) = \sum_{j=0}^{r-1} j \ell(s_j).$$

A transfer reduces Φ by exactly one before any deletion. Removing an identity or extracting an initial half-twist cannot increase it. The involution τ preserves simple length. Appending the k th input letter increases Φ by at most $B(k-1)$, and creates at most one factor. Starting from zero potential proves the transfer bound by summing these increases.

A transfer or a test for a transferable atom scans at most b positions. The scan index advances on a failed test and moves back by at most one after a transfer, so the number of tests is bounded by the number of scanned factors plus a constant multiple of the transfers. Over the entire word, there are at most m factor deletions and m half-twist extractions; even literal Python-list shifting contributes only $O(m^2)$ operations. Copies and applications of τ cost $O(m^2b)$ in total. Since $B = O(b^2)$, the stated $O(m^2b^3)$ bound follows.

For a single append, the initial and appended list has at most $r+1$ factors, so its potential is $O(B(r+1)^2)$. List shifts contribute $O((r+1)^2)$, and the same test accounting gives the second bound. The half-twist exponent and factor indices grow at most linearly with input length; strand positions are at most $b-1$. $\square$

This bound is intentionally conservative. It establishes a polynomial exact engine for the new optimizer; it is not a claim that this implementation matches the best known Garside algorithms.

3.3 Canonical height and interval states

Let $\ell_c(g)$ denote the number r in (1), ignoring the half-twist power. For the doubled word ww , let P_j be its prefix product, and set

$$R = \max_{0 \leq j \leq 2n} \ell_c(P_j). \quad (2)$$

Always $R \leq 2n$; it can be much smaller.

Lemma 3.3 (Height calculus). *For all braids g, h ,*

$$\ell_c(g^{-1}) = \ell_c(g), \quad \ell_c(gh) \leq \ell_c(g) + \ell_c(h).$$

Thus every interval prefix $P_i^{-1}P_j$ of ww has canonical height at most $2R$, and every target query P_jv^{-1} with $|v| \leq \rho$ has height at most $R + \rho$.

Proof. To multiply two normal forms, commute the second half-twist power to the left, applying the appropriate power of τ to the first factor list, and concatenate the lists. Normalization does not increase the number of factors. To invert a simple factor, write its inverse as a half-twist inverse times a simple complement. Applying this to a list gives an inverse representation with at most the original number of simple factors. Applying the same inequality to g^{-1} yields equality. The remaining claims follow by subadditivity and $\ell_c(v^{-1}) \leq |v|$. $\square$

4 Shared prefixes and an exact cyclic optimizer

4.1 The endpoint identity

For any original or doubled-word interval,

$$w[i : j] = v \iff P_i = P_jv^{-1}. \quad (3)$$

This is an equality of braid elements, not a conjugacy test. It holds for all supports. In particular, it must not be replaced by equality of permutations: pure braids would then produce false cancellations.

Compute the complete canonical states of $P_0, \dots, P_{2n}$. Sort these states lexicographically, remove exact duplicates, and assign integer identifiers. Look up every state P_jv^{-1} by binary search in that same sorted table. If the state is absent, that target cannot match any start endpoint in the doubled word. This interning is deterministic and requires no expected-time hash-table hypothesis.

For a cut s , relative interval positions i, j refer to global positions $s+i, s+j$. Their products are still $P_{s+i}^{-1}P_{s+j}$. Consequently the same global query table works for *every* cut. Recomputing a normal form of every rotated word is unnecessary; no equation requires conjugating a query state or cyclically reducing its normal form.

4.2 The scalar dynamic program

Fix a cut, and write $F[j]$ for the maximum number of letters removed from the first j letters of that rotation. Immediately before processing j , maintain for each canonical-state identifier q

$$H(q) = \max_{0 \leq i < j : P_{s+i}=q} (F[i] - i), \quad (4)$$

with one attaining i . Missing entries have value $-\infty$. Then

$$F[j] = \max \left\{ F[j-1], \max_{v \in D} (j - |v| + H(P_{s+j}v^{-1})) \right\}. \quad (5)$$

After all queries at j , insert $F[j] - j$ under the identifier of P_{s+j} . The query-before-insertion order excludes empty source intervals.

Theorem 4.1 (Exact one-pass optimization). *The recurrence (5), with backpointers to attaining starts and targets, returns $\kappa_D^{\text{lin}}(\text{rot}_s(w)) = n - F[n]$. Taking the best of all cuts returns $\kappa_D^{\text{cyc}}(w)$. Only strictly shorter replacements need to be recorded.*

Proof. An optimal plan on a prefix either retains its last letter, contributing at most $F[j-1]$, or ends in a replacement $[i, j] \mapsto v$. In the latter case, all earlier replacements lie in the first i letters, so its savings are at most $F[i] + j - i - |v|$. Equation (3) and the maximum in (4) enumerate precisely these possibilities. Conversely, each term of the recurrence extends a valid optimal prefix plan, so induction proves equality.

If $j - i \leq |v|$, its candidate savings are at most $F[i]$, and hence at most $F[j-1]$. Such a candidate cannot strictly improve the retained-letter choice. Choosing backpointers only on a strict improvement therefore records only strictly shortening source intervals. Backtracking jumps from j to its chosen start i , so the recorded intervals are disjoint.

The outer minimum is the definition of the cyclic optimum. Equivalently, any disjoint plan drawn on a circle can be cut at a boundary of one of its blocks (or at any unused boundary), and is represented by one of the n linear problems. A block covering the entire word also has a boundary. $\square$

For radius one this extends the repository's prefix recurrence from its faithful rank-two local states to exact global states. The innovation is not the last-interval induction itself, but the unrestricted, shared realization of its group queries and the associated verifiable cost model.

4.3 Sharing the target arithmetic as well

For a general finite dictionary, build a trie of the literal words v^{-1} . Starting with P_j at the root, append one signed letter along each trie edge. The state at the terminal for v^{-1} is $P_j v^{-1}$. Each trie edge is traversed once at that endpoint. The same partial product is not recomputed separately for all targets extending it.

The implementation constructs a deterministic depth-first trie by sorting inverse target words and computing longest common prefixes. Only states along the current trie branch need remain live. For the full radius- ρ word ball, every prefix is itself in the dictionary, so the number of trie nodes is exactly M_ρ .

Theorem 4.2 (Shared algebra count). *For a nonempty word and a target trie with T nodes, including the root, the all-cut preprocessing uses exactly $2nT$ calls to the signed-atom append operation: $2n$ for doubled prefixes and $2n(T-1)$ for target edges. This excludes generation of the final winning certificate.*

A reference implementation that freshly prepares n prefixes and the same trie for each of the n rotations uses n^2T such calls. Both still perform $O(n^2|D|)$ scalar dynamic-programming work in the worst case.

Proof. The doubled word has $2n$ nonempty prefixes. At each endpoint its root state is already available, and every nonroot trie node requires exactly one append from its parent. These

computations do not depend on the cut. The reference repeats the analogous $n + n(T - 1) = nT$ construction for each cut. $\square$

The ratio of the two append counts is exactly $n/2$, not a general elapsed speedup theorem. States for doubled prefixes can be taller than states for individual rotations, scalar work is unchanged, and the doubled setup can lose on the smallest inputs. Safe lower-bound stopping can reduce the number of evaluated cuts, but the current implementation prepares the entire query table first.

5 Bit complexity, storage, and certificate size

Theorem 5.1 (Height-sensitive kernel bound). *Let $n \geq 1$, $b \geq 2$, $\rho \geq 1$, $M = M_\rho$, and let R be (2). The full radius- ρ cyclic optimizer, including a winning certificate and its independent replay, has the following conservative word-operation bound:*

$$T(n, b, \rho, R) = O(nM(R + \rho + 1)^2 b^3 + nM(R + \rho + 1)b \log(n + 2) + n^2 M + M\rho \log(M + 2) + n(R + \rho + 1)b^3). \quad (6)$$

Its search workspace, excluding the emitted proof, is

$$O(n(R + 1)b + nM + M\rho + \rho(R + \rho + 1)b) \quad (7)$$

words. A winning residual-word certificate has $O(n(R + \rho + 1)b^2)$ elementary records, and replay costs $O(n(R + \rho + 1)b^3)$ word operations.

All computations admit a bit-cost bound obtained by multiplying (6) by $O(\log(n + b + \rho + M + 2))$. For each fixed ρ , the worst-case preprocessing is polynomial in n, b . In particular, the leading conservative word bounds are $O(n^3 b^4)$ for radius one and $O(n^3 b^5)$ for radius two, with the logarithmic lookup and bit factors understood as in (6).

Proof. There are $O(nM)$ signed-atom append calls. By Lemma 3.3, the temporary query states have at most $R + \rho + 1$ factors. The single-append bound of Theorem 3.2 gives the first term of (6).

Comparing two canonical states scans at most $O((R + \rho + 1)b)$ integer entries. Sorting the original prefix table and binary-searching the target states gives the second term, which also dominates the original-prefix sort. Arrays for (4) and the backpointers require $O(n)$ initialization per cut and $O(nM)$ target checks per cut, giving $O(n^2 M)$. Generating and sorting the target dictionary and its trie accounts for $O(M\rho \log(M + 2))$.

For a recorded interval u with target v , version-two certificates normalize uv^{-1} to the identity. The source intervals are disjoint, so $\sum |u| \leq n$. Strict shortening implies $\sum |v| < \sum |u|$ when there are replacements. Thus the total number of residual letters is at most $2n$. During the u portion, every relative prefix has height at most $2R$. During the target inverse portion it has height at most $2R + \rho$. The potential proof now bounds the total transfers by the sum of the per-append increases, $O(n(R + \rho + 1)b^2)$, rather than by the square of the largest interval length. Resetting to the identity between intervals introduces no positive initial potential. Testing and replaying each transfer costs $O(b)$; copies and list deletion shifts are covered by the same bound. This proves the certificate and replay assertions and the last term in (6).

The original canonical states use $O(n(R + 1)b)$ words. The query table has $O(nM)$ integer entries, the dictionary and trie use $O(M\rho)$ words, and only a depth- ρ stack of query states need be retained during a trie walk. The dynamic-programming arrays are smaller than these

terms. This proves (7). The proof itself is an additional output allocation, not included in that search-workspace claim.

Permutation positions, powers, source indices, target identifiers, and counters fit in words of the displayed logarithmic bit length. Binary comparison, addition, and array-address bookkeeping can be implemented within that bit factor. With $R \leq 2n$ and fixed ρ , $M = O_\rho(b^\rho)$, which gives polynomial worst-case cost and the two stated leading terms. $\square$

Remark 5.2 (Parameter accounting). The polynomial claim does not cover an arbitrary radius supplied in binary. The target dictionary is exponential in $\rho \log(b+1)$. For a varying radius, (6) is the operative bound. In particular, a radius that makes M quasi-polynomial can still yield quasi-polynomial preprocessing, but not polynomial preprocessing by assertion. The finite-radius API has an explicit target-count cap; the theorem concerns the uncapped route.

5.1 Why a cheap whole-word candidate comes first

The optimizer solves a restricted combinatorial objective. A classical normal form may directly expose a short equal word outside that objective. For a normal state (1), expand the half-twist power and each simple factor to a reduced positive word. Its length is

$$|z| \binom{b}{2} + \sum_i \ell(s_i).$$

The code computes this length before allocating the expansion and keeps it only when it is shorter than the input. Equality is checked by a residual certificate, just as for a dictionary replacement.

Proposition 5.3 (Certified candidate portfolio). *Compute this whole-word candidate first. If it attains $L(w)$, return it. Otherwise compute the radius- ρ cyclic kernel on the original input and return the shorter verified candidate. The portfolio output length k satisfies*

$$k \leq \kappa_\rho^{\text{cyc}}(w),$$

and its closure is isotopic to the original closure. It adds only polynomial normalization and replay overhead. It need not itself be an optimal radius- ρ plan.

Proof. If the first candidate attains the universal lower bound, its length is no larger than the kernel optimum. Otherwise the kernel is explicitly one of the compared candidates. The whole-word candidate is equal to the original braid, while the cyclic candidate is equal after its recorded rotation. Both certificates are rooted at the original word, so no unrecorded composition of transformations is needed. $\square$

This shortcut is classical algebra used responsibly, not a new Garside theorem. It is particularly important in interpreting the radius separation below: that family is difficult for a specific replacement dictionary, not for every braid simplifier.

6 Independent replay and the trust boundary

6.1 What is recorded

A certificate records the strand count, a source digest, the linear or cyclic scope, the rotation, the output, and disjoint original-coordinate source intervals. Each interval carries its target

and a per-input-letter sequence of simple-factor transfer locations and extracted half-twist counts. Version one specializes targets to the empty word or one signed letter; version two checks the residual word uv^{-1} against the identity.

The verifier does not import the optimizer or the normalizer. It reconstructs its own permutation arrays, tests the two inequalities in Lemma 3.1 before every transfer, and checks every half-twist extraction literally. Negative input letters use the explicit complement and τ formulas, not an asserted intermediate normal state. A successful version-two replay must end with exponent zero and no simple factors.

Theorem 6.1 (Replay soundness). *Every accepted local transcript proves equality of its actual source interval and target. Every accepted full certificate proves equality after the recorded rotation and hence equality of the oriented closure type. This soundness statement does not require the verifier to establish maximality of a normal form or optimality of the dynamic program.*

Proof. The replay begins with the identity expression. Each input-letter update is an exact braid identity, each legal transfer is Lemma 3.1, and each extraction merely rewrites a leading Δ as a change to its integer power. Induction therefore identifies the final explicit expression with the residual braid. Ending at the identity proves $uv^{-1} = 1$. Disjoint intervals may then be replaced in their original surrounding contexts. Finally a single cyclic cut changes a braid by conjugacy, which preserves its closure. $\square$

A dishonest optimizer can fail to find an improvement or provide a rejected transcript, but cannot turn a legal replay into a false braid equality under the mathematical model. Tests support the implementation of that model; they are not a formal proof of Python interpreter correctness.

6.2 Checks and limitations

The verifier checks exact field sets, integer types excluding booleans, valid generator ranges, a valid cut, increasing nonoverlapping intervals, strict shortening, proof lengths, local move legality, and exact output reconstruction. A source digest associates a transcript with an input; it is not an authenticity signature, and the proof still acts on the actual supplied source word. A certificate proves only the transformations it contains. It does not prove that all cuts were searched, that a radius was saturated, that the output is a minimum braid, or that its closure is the unknot.

Operation allowances are cooperative. Search checks a tick counter during normalization and dynamic programming; verification can bound the number of transferred atoms. These are not hard wall-clock or process-memory limits. JSON parsing, dictionary allocation, and ordinary list copies have costs beyond a single tick. A robust host should add process-level limits when needed. Allocation failure, exhausted allowance, or an invalid certificate must never become a knot verdict.

The small independent Artin-action oracle expands free-group words and can have exponential intermediate size. It is used only for bounded tests and never appears in the polynomial-time production route.

7 An infinite strict separation of target radii

Here is an obstruction to *our own* unrestricted cyclic radius-one kernel. It demonstrates why removing the support restriction alone is not a sufficient structural theorem.

Write $a = \sigma_1$, $b = \sigma_2$, and $c = \sigma_3$ in B_4 . For $m \geq 2$, define

$$X_m = a^{-m}c^m a^{m+1}c^{-(m-1)}, \quad W_m = X_m b. \quad (8)$$

The commutation $ac = ca$ gives $X_m = ac$, so $W_m = acb$ as a braid, not just as a closure. Its literal length is $4m + 1$.

Theorem 7.1 (Unbounded radius-one/radius-two separation). *Every W_m closes to the unknot, and*

$$\boxed{\kappa_1^{\text{cyc}}(W_m) = \kappa_1^{\text{lin}}(W_m) = 4m + 1, \quad \kappa_2^{\text{cyc}}(W_m) = \kappa_2^{\text{lin}}(W_m) = 3.}$$

In fact no strictly shortening radius-one substitution is available at any cyclic cut. Repeated radius-one passes therefore make no progress.

We prove this with a lattice path and a double-coset calculation.

Lemma 7.2 (An induced commuting path). *The prefix products of X_m in $P = \langle a, c \rangle \cong \mathbb{Z}^2$ form a simple induced path in the unit square lattice from $(0, 0)$ to $(1, 1)$. No nonempty subinterval is the identity, and no subinterval of length greater than one equals $a^{\pm 1}$ or $c^{\pm 1}$.*

Proof. The four successive axis-aligned segments run through

$$(0, 0), \quad (-m, 0), \quad (-m, m), \quad (1, m), \quad (1, 1).$$

They do not intersect except at consecutive corners. Adjacent segments meet monotonically at a right angle, so any two vertices on them have lattice distance equal to their path distance through the corner. Nonadjacent horizontal segments are separated by $m \geq 2$, and the two vertical segments by $m + 1 \geq 3$. The remaining bottom/last-segment pair has distance at least two, since its bottom coordinate has $x \leq 0$ and the last segment has $x = 1, y \geq 1$. Thus no nonconsecutive vertices have lattice distance one. Equal prefixes would be a repeated vertex, and a long single-generator interval would be a forbidden lattice edge. $\square$

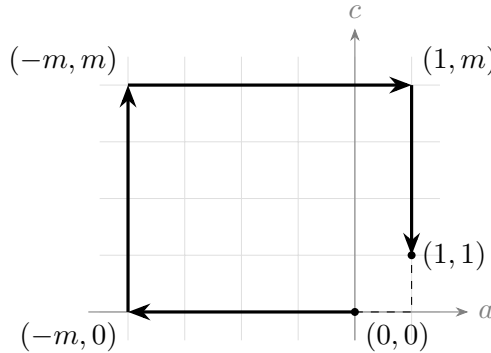


Figure 1: The commuting detour X_m (shown at $m = 4$). The dashed two-letter path is ac . Every long subpath avoids an identity or single-letter shortcut, although the whole detour equals this two-letter path.

Lemma 7.3 (The middle-generator obstruction). *For $p, q \in P = \langle a, c \rangle$, the braid pbq cannot be the identity or $a^{\pm 1}$ or $c^{\pm 1}$. It equals a signed middle generator only in the case*

$$pbq = b, \quad p = q = 1.$$

In particular it never equals b^{-1} .

Proof. At permutation level let $H = \langle (12), (34) \rangle$ and $t = (23)$. The double coset HtH is disjoint from H , proving the first assertion. If $pbq = b^{\pm 1}$, then the permutations of p and q lie in $H \cap tHt^{-1} = \{1\}$. Thus write $p = a^{2r}c^{2s}$ and $q = a^{2u}c^{2v}$.

Track signed crossings between the original labeled strands. This vector is invariant under braid relations: cancelling opposite crossings gives zero; a commuting relation preserves its two labeled pairs; and the braid relation crosses the same three labeled pairs once on either side. For a pure power a^{2r} only pair 12 has count $2r$, and analogously c^{2s} contributes to pair 34. Since the permutation of p is the identity, the middle letter b contributes $+1$ to pair 23. After that letter, the contributions from q are $2u$ on pair 13 and $2v$ on pair 24. Equality to $b^{\pm 1}$ therefore forces $r = s = u = v = 0$ by the four other pair counts, and the count on pair 23 forces the positive sign. $\square$

Proof of Theorem 7.1. A cyclic source interval not containing the unique letter b lies in the path X_m . Lemma 7.2 excludes identity and long single-letter replacements there; a word in P cannot equal $b^{\pm 1}$ by permutation. An interval containing b has the form pbq , where p is a suffix of X_m and q is a prefix. Lemma 7.3 permits a signed-letter target only when $p = q = 1$. Simplicity of the prefix path says that no nonempty prefix of X_m is the identity and no nonempty suffix has identity product. Thus the only such interval is the original single letter b , which does not strictly shorten. This proves the radius-one assertion at all cuts.

Radius two may replace X_m by ac , leaving acb of length three. Its permutation is a four-cycle, so Lemma 2.2 forbids an output shorter than three. Finally the closure of acb is the unknot: rotate to cba , remove the singleton left endpoint generator a by Markov destabilization, obtaining a three-braid $\sigma_2\sigma_1$; rotate and destabilize again, and then remove the remaining two-strand crossing. Every step preserves the oriented closure type. $\square$

Remark 7.4 (What this separation does not prove). The family has a three-letter *equal braid representative*. A whole-word Garside normal form immediately exposes a short simple braid. Accordingly our candidate portfolio handles it before the larger dictionary is built. The theorem separates the radius-one and radius-two *interval rules*; it is not a lower bound for all braid algorithms, all Garside reductions, or all unknot recognizers. This distinction is also visible in the timing table in Section 10.

8 Further obstructions and the role of cyclic closure

8.1 Removing the repository's rank-two blind spot

The maintained rank-two analysis supplies the family [5]

$$U = abcaba, \quad V = cbacbc, \quad Q_h = (UV^{-1})^h abc \in B_4.$$

Both U and V are positive words for Δ_4 . Hence $Q_h = abc$, its closure is the unknot, and its length is $12h + 3$.

Proposition 8.1 (Unrestricted support removes that obstruction). *For every $h \geq 1$,*

$$\kappa_1^{\text{lin}}(Q_h) = \kappa_1^{\text{cyc}}(Q_h) = 3.$$

The original rank-two strictly shortening rule set cannot begin on Q_h , even if it permits longer target words.

Proof. Our rule set deletes the whole identity prefix $(UV^{-1})^h$. The output abc has a four-cycle permutation, forcing the lower bound three. For the rank-two obstruction, any subword lying

inside a uniform-sign block is already geodesic by exponent sum. A subword using at most two indices and crossing a sign boundary is confined to ac^{-1} or $c^{-1}a$, because the neighboring letters have index b . These two-letter words have two independent commuting exponents and cannot shorten. $\square$

This is an exact extension of the repository's compression capability. It is not an end-to-end performance claim: the maintained analysis reports that an earlier descending-diagram certificate already settles its barrier fixture, and whole-word normalization also exposes the short braid. A new preprocessing stage should not displace a cheaper certificate merely to show a favorable isolated benchmark.

8.2 Geodesic braid representatives of the unknot

There is a more fundamental obstruction to any method that insists on preserving the original braid element while seeking a short word.

Theorem 8.2 (A geodesic unknot family). *In $B_3 = \langle a, b \mid aba = bab \rangle$, let*

$$G_m = a^{m+1}ba^{-m} \quad (m \geq 1).$$

Its closure is the unknot, but its minimum Artin word length as the same braid element is exactly $2m+2$. Consequently no strictly shortening braid-equality substitution, with any support or any target length, can start on any subword of G_m . Nevertheless,

$$\kappa_1^{\text{lin}}(G_m) = 2m + 2, \quad \kappa_1^{\text{cyc}}(G_m) = 2.$$

Proof. Put $x = aba$, $y = ab$. The presentation becomes $\langle x, y \mid x^2 = y^3 \rangle$. Quotienting by the common central power gives the free product $C_2 * C_3$. In this quotient,

$$a = y^2x, \quad a^{-1} = xy, \quad b = xy^2, \quad b^{-1} = yx.$$

The reduced free-product word for G_m is

$$(y^2x)^m y (xy)^m,$$

which has $4m+1$ syllables. The image of each signed Artin letter has two syllables before reduction, so every word equal to G_m has length at least $\lceil (4m+1)/2 \rceil = 2m+1$. Exponent sum is two; word length must therefore be even. The lower bound improves to $2m+2$, attained by the given word. A shortening of a subword would shorten the whole equal braid, contradicting this geodesicity.

On the other hand, $G_m = a^m(ab)a^{-m}$ is conjugate to ab , whose closure is the unknot by two destabilizations. Rotating after the first m letters exposes $aba^{-m}a^m$, and the final identity block may be removed. Its three-cycle permutation forbids an output shorter than two. $\square$

Corollary 8.3 (A limitation on equality-only small-core programs). *No preprocessor whose output must be equal to the original braid element can guarantee an $O(\log^C n)$ -letter output on every unknot braid, for any fixed C . This remains false at the fixed strand count three.*

Proof. Apply the geodesic lower bound to $n = 2m + 2$. $\square$

This does not exclude efficient algorithms operating on compressed objects, and it does not exclude conjugacy or Markov transformations. In fact the repository already performs free cyclic cancellation [6], so it can remove this particular conjugating sleeve without the new global optimizer. The theorem isolates a logical requirement, not an unclaimed new speedup on that input.

8.3 Hidden sleeves beyond literal free cancellation

For another test family, let

$$u_m = (aba)a^m, \quad v_m = (bab)a^m, \quad Z_m = u_m(ab)v_m^{-1}.$$

Since $u_m = v_m$, a cyclic cut followed by the identity replacement $v_m^{-1}u_m \mapsto \varepsilon$ proves $\kappa_1^{\text{cyc}}(Z_m) = 2$. For $m \geq 2$ the written word is freely and cyclically reduced: no adjacent inverse pair occurs even at the circular seam. Thus free cancellation alone does not expose every sleeve that the global prefix method finds.

For $2 \leq m \leq 16$, the supplied tests additionally find $\kappa_1^{\text{lin}}(Z_m) = |Z_m|$. This last statement is reported only as a finite observation, not as a theorem for all m . The family lies in three strands, so a complete closed-three-braid gateway may already decide its closure; again no complete-recognizer gain is inferred from the kernel output.

Even a four-letter three-braid can be a cyclic radius-one fixed point: $ba^{-1}ba$ is conjugate to ab by $a^{-1}b^{-1}$, and the tests verify $\kappa_1^{\text{cyc}} = 4$, $\kappa_2^{\text{cyc}} = 2$. The infinite four-strand separation in Theorem 7.1 supplies the stronger, unbounded version with a uniform proof.

9 A constructive quasi-polynomial recognition class

9.1 The exact fallback cost

Lemma 9.1 (Conventional exact fallback). *A k -crossing braid diagram with a knot closure admits an exact unknot recognition algorithm using $\text{poly}(k)2^{O(k)}$ bit operations.*

Proof. One route is to construct the reduced rational Khovanov complex and compute its total homology rank. Its cube has 2^k resolutions. For a b -strand braid the number of circles in a resolution is at most $b + k$, and for a knot closure $b \leq k + 1$. Thus the total number N of enhanced-state basis vectors is $2^{O(k)}$. The differential matrices have entries $0, 1, -1$.

Exact rational ranks can be computed by fraction-free elimination or an exact minor-based elimination scheme using a polynomial number of operations in N . Hadamard's determinant bound gives $O(N \log(N + 2))$ bits for the relevant integer minors of these matrices. Therefore the full bit complexity is still $N^{O(1)} = 2^{O(k)}$, with only polynomial factors in k . The chain property gives homology dimensions from the matrix ranks. Reduced rational Khovanov homology has rank one exactly for the unknot by Kronheimer–Mrowka [2]. $\square$

This lemma supplies a concrete theoretical fallback, not an unexplained exponential oracle. The maintained repository uses its own exact homology pipeline. The distributed new code is a preprocessor and verifier; it does not reimplement that entire backend or claim that an external smoke test has been run.

Theorem 9.2 (Small-kernel recognition). *Fix a radius ρ . For a braid input whose closure is a knot, run the uncapped certified radius- ρ kernel or the portfolio of Proposition 5.3, then use an uncapped exact fallback as in Lemma 9.1. If its output has k letters, the total time is*

$$\text{poly}_\rho(n, b) + \text{poly}(k)2^{O(k)}.$$

In particular, the algorithm is quasi-polynomial on every family satisfying

$$\kappa_\rho^{\text{cyc}}(w) = O(\log^2(n + 2)) : \quad T(n) = 2^{O(\log^2(n+2))} = n^{O(\log n)}$$

for nontrivial growing input sizes.

Proof. Theorems 4.1, 5.1, and 6.1 provide the polynomial preprocessing and preservation of closure type. For the portfolio, its output is no longer than the kernel optimum. The output permutation remains a single b -cycle, so $b \leq k + 1$. Diagram construction adds polynomial cost, and Lemma 9.1 gives the remaining term. Substitution of the stated bound on k proves the quasi-polynomial conclusion. $\square$

9.2 A class whose decomposition is not supplied

Definition 9.3 (Conjugated flat radius- ρ inflation). Take a core braid word c of length k_0 on the same strands and partition it into consecutive blocks of length at most ρ . Replace each block by a no-shorter word equal to it in the braid group; insert any number of disjoint identity blocks. Call the result y . Finally form

$$w = yv, \quad vu = 1 \text{ in } B_b.$$

Neither the partition, the equalities, nor the two sleeve words need be provided to the optimizer.

Corollary 9.4 (Constructive inflation theorem). *Every such word satisfies $\kappa_\rho^{\text{cyc}}(w) \leq k_0$. Hence, when its closure is a knot and $k_0 = O(\log^2(n + 2))$, it is recognized in quasi-polynomial time by the uncapped composition, without a supplied inflation decomposition.*

Proof. Rotate to yvu . Delete the identity sleeve vu , replace the inflated blocks by their core targets, and remove the inserted identity blocks. These are disjoint permitted intervals. Blocks that were not lengthened can simply be retained, so only strictly shortening substitutions are needed. The plan leaves c . Exact optimization finds an output at least as short as this existing plan without needing to be given it. $\square$

Radius one permits arbitrarily high-support inflation words for individual letters, extending the previous rank-two setting. Radius two also permits commuting detours $X_m \mapsto ac$ inside a core, including the strict separation family. Conjugated versions and disjoint combinations are covered automatically. No claim is made that every unknot diagram admits one of these decompositions with a small core.

9.3 Where the remaining global gap is located

The theorem is not just a promise that an already supplied short certificate will be checked: the decomposition is searched for in polynomial time. Nevertheless, the family hypothesis bounds the output of an explicit restricted optimization problem. It is not a theorem that the output is small for arbitrary knot inputs. The geodesic family excludes a universal equality-only word-length argument; the rectangle family excludes a universal radius-one interval argument even with all cyclic cuts.

Additional closure-preserving operations, such as independently certified Markov moves or exchange moves, could enlarge the class. Their search cost, the number of required moves, and every intermediate representation size must all be controlled. Merely naming such transformations does not close the quasi-polynomial proof.

10 Implementation validation and measurements

10.1 Reproducible finite checks

The delivered test run contains **47 passing tests** on CPython 3.13.5. The retained log records 11.952 seconds for that run; wall times will vary. Within these tests are the following independent or exhaustive checks.

The normal-form engine is compared with the faithful Artin action on all 5,461 three-braid words through length six and all 1,555 four-braid words through length four: 7,016 words in total, with collision consistency checked in both directions. All 872 permutations on two through six strands are checked as simple braids. Additional seeded tests check 500 independently expanded normal forms, 300 inverse products, defining relations in 600 contexts, and 100 transfer-potential bounds.

The radius-one optimizer is compared against an exhaustive interval oracle in 1,162 linear/-cyclic instances. The radius-two optimizer adds 350 such comparisons, for **1,512 independent optimum comparisons**. These include repeated or empty inputs where the test definitions generate them; the count is a number of comparisons, not a count of distinct knots. The generic radius-one trie implementation is also compared with the specialized implementation in 300 instances.

Other tests cover rotation equivariance, exact lower-bound stopping, conjugated inflations, the explicit infinite-family formulas at bounded parameters, portfolio dominance in 240 radius-specific comparisons, source association, invalid indices and booleans, overlapping intervals, altered targets, corrupted proof lengths, and exhausted local allowances. The tests do not establish an asymptotic theorem by extrapolation; those bounds rest on the preceding proofs.

10.2 A paired test of shared versus duplicated algebra

The primary benchmark compares the shared doubled-prefix engine with a reference that prepares fresh prefix algebra for every cyclic cut. Both use the same normal-form implementation, target dictionary, and scalar dynamic program. The reference generates and replays only the winning certificate, rather than wasting proof generation at every rotation. Safe lower-bound stopping is disabled in *both* arms to isolate the all-cut construction. Imports and input generation are excluded; winning certificate generation and independent replay are included.

There are seven shuffled paired rounds per case, with two identical shared arms A and B as an A/A control. Inputs, seeds, per-run times, counters, verdict-free outputs, and environment details are retained in `data/benchmarks.json`. The host reports Linux x86-64 and an AMD EPYC 9V74 processor. No arm was truncated by a timeout.

The six multi-crossing fixtures show median paired ratios from about 4.3 to 20.6; the one-crossing fixture shows overhead, with a ratio below one. The A/A controls are near one but do not constitute a statistical confidence interval. Exact append counts match Theorem 4.2; elapsed ratios differ because height and scalar work also matter. The input corpus is deliberately small and structured, not a representative distribution of hard knots.

A separate seven-round run uses the default lower-bound stopping on the repository's Q_h family. It evaluates only the first cut and attains the minimum length three. Canonical doubled-prefix height remains two on the recorded cases.

Input	n	k	Shared (ms)	Naive (ms)	Ratio	A/A
single_crossing	1	1	0.02	0.02	0.87	1.03
positive_4_braid	9	9	0.62	2.64	4.30	1.05
ranktwo_barrier_h1	15	3	1.50	10.66	7.56	0.95
ranktwo_barrier_h4	51	3	5.89	121.08	20.56	1.02
geodesic_m12	26	2	10.35	61.90	5.93	1.01
hidden_sleeve_m8	24	2	7.83	50.68	6.33	0.95
seeded_5_braid	22	6	9.61	89.04	9.27	1.01

Table 1: Compressor-level all-cut measurements. Times are median milliseconds; Ratio is the median paired naive/shared-A ratio; A/A is the median paired shared-B/shared-A ratio. These are not complete-recognizer timings.

h	n	k	R	Time (ms)	Certificate bytes
8	99	3	2	8.65	3324
32	387	3	2	34.83	12589
128	1539	3	2	136.72	49646

Table 2: Certified radius-one kernel on Q_h , with the exact lower-bound shortcut enabled. Certificate sizes are compact JSON bytes. The separate whole-word candidate is not included in these kernel timings.

10.3 Radius enlargement has a real cost

The next experiment checks the theorem $\kappa_1^{\text{cyc}}(W_m) = 4m + 1$ versus $\kappa_2^{\text{cyc}}(W_m) = 3$ on the rectangular family. It also measures the practical portfolio, whose whole-word normal-form candidate already attains three. The two radius arms were shuffled in seven paired rounds. Portfolio times come from a separate seven-round run and are descriptive medians, not a paired speedup estimate. All three measurements include their required independent replay.

m	n	κ_1	κ_2	Radius 1	Radius 2	Portfolio
2	9	9	3	1.24	7.69	0.22
8	33	33	3	15.67	94.82	1.91
32	129	129	3	229.99	1369.98	28.78

Table 3: Rectangular-family outputs and median times in milliseconds. The two kernel arms solve different optimization problems. The portfolio returns a three-letter equal braid without constructing the larger target query table.

Radius two pays substantial dictionary and normal-form overhead even though it finds a dramatically shorter kernel. On the 129-letter example its median is about 1.37 seconds, versus about 0.23 seconds for an unchanged radius-one kernel. The cheap portfolio instead takes about 29 milliseconds. No elapsed full-recognizer gain follows from those numbers: an upstream structural certificate might already recognize the knot faster. Conversely, comparing unchanged and shortened kernel times alone ignores the possible cost of a later exact homology computation. Both facts are why end-to-end measurements are required before changing defaults.

10.4 What has not been executed

The complete ProveIt test suite was not run in this environment. The repository was inspected through the GitHub connector at the recorded snapshot, but a full local checkout was not available. Thus the upstream suite’s reported 272 passing tests are source-reported status, not a claim of a new run here [4]. The optional supplied upstream smoke script is marked unexecuted. No Rust integration or Lean formalization is claimed, and no public repository files were modified.

11 Integration into the maintained recognizer

11.1 Placement and provenance

The proposed package is additive and optional. Keep existing inexpensive certificates, free cyclic cleanup, singleton endpoint destabilization, and the specialized rank-two stage ahead of a potentially expensive global probe. Exact whole-word normalization is the first stage of the proposed portfolio; an unrestricted cyclic kernel follows only if needed.

The current `Diagram.braid_source` property identifies a checked source braid; it is not an unchecked auxiliary input. A probe must operate on that source, independently replay its candidate, and construct a replacement `Diagram.from_braid` only when it improves on the *current* simplified diagram’s crossing count. Otherwise it should retain the existing pipeline state. Earlier planar-diagram reductions and a restarted source braid belong to different evidence branches. A cyclic braid certificate connects them through the original source, not by pretending to be a concatenated Reidemeister move trace.

The supplied optional adapter follows this pattern. It is not wired into the maintained recognition function. It supplies a candidate and evidence, not a knot verdict. An adopter must preserve the recognizer’s remaining global budget, avoid recursively re-enabling the same probe on restart, and retain all existing fallback and provenance rules.

11.2 Failure behavior

A local search limit or dictionary cap is inconclusive: discard the probe and continue from the unchanged input. Global budget exhaustion remains `UNKNOWN` under the host recognizer’s contract. An invalid replay is an implementation error to investigate, never evidence for knot or unknot. The conditional complexity theorem applies when the relevant local and fallback caps are removed; bounded production runs can return inconclusive results and are not silently substituted for the theorem’s algorithm.

The candidate portfolio uses only a single certificate rooted at the original word. If future versions repeatedly alternate diagram reduction, normalization, and cyclic kernels, a provenance graph or explicit sequence of source-to-source certificates will be necessary. Those additional passes must be counted separately.

12 Further research questions

1. A strict hierarchy beyond radius two. For each fixed ρ , construct and prove an infinite unknot family with $\kappa_\rho^{\text{cyc}}(w) = \Omega(|w|)$ but $\kappa_{\rho+1}^{\text{cyc}}(w) = O(1)$, or determine where such a statement fails. The lattice construction proves the first step. A stronger and more practically

relevant version should remain difficult after whole-word normal-form expansion and cheap Markov descent. Our rectangle family does not meet that stronger condition.

2. Reuse scalar work between adjacent cuts. Theorem 4.2 removes repeated braid algebra, not the $O(n^2M)$ scalar term. Can the interval-state structure support a rolling maximum, persistent dynamic program, or a specialized min-plus algorithm that reuses more work between cuts? Any proposed update must handle the changed left boundary and the loss of old maximizing starts exactly.

3. Persistent canonical states with a lazy half-twist action. The prototype copies simple-factor lists and applies τ eagerly on negative input. Persistent trees, lazy involutions, and local left-weighting may reduce this overhead. A useful next theorem would bound update work in terms of the actual changed normal-form suffix, with a deterministic comparison and memory model rather than unverified hash equivalence.

4. Implicit targets for commuting powers and parabolic blocks. A literal radius ball is expensive. Can families such as $a^p c^q$, short geodesics in a small parabolic subgroup, or simple braids be queried without enumerating all target words? The output cost must remain actual Artin length or another explicitly justified objective. The empty-target and one-letter table is insufficient for the rectangular detours.

5. Closure-aware operations with controlled search. Combine the portfolio with certified destabilization, exchange moves, conjugation changes not visible as literal cuts, and selected length-preserving relations. Identify families on which the number of rounds and all intermediate sizes are bounded. The geodesic theorem explains why equality-only kernels cannot be the whole strategy.

6. A structural bridge to the quasi-polynomial target. Prove that a meaningful, naturally described class of unknot diagrams admits a polynomially constructible sequence leading either to a $O(\log^2 n)$ kernel or to a compressed exact-homology calculation with a quasi-polynomial size bound. No universal bound on braid index, scan multiplicity, or kernel size should be inserted without proof.

7. Proof compression and formal verification. Formalize the simple-transfer invariant and residual replay theorem, then the last-interval optimality induction. Compress repeated transfer segments using a checked straight-line proof language. Such compression must preserve a polynomial verifier and must not move an unbounded normal-form computation into the trusted checker unnoticed.

8. Complete-recognizer evaluation. Run the optional stage against a pinned checkout, preserving cheap certificates and time accounting. Use shuffled paired end-to-end runs, A/A controls, memory measurements with censoring reported, and a corpus including cases where each probe fails. Report output-size improvement separately from actual time-to-verdict. The included local data are a starting point, not a substitute for this evaluation.

13 Conclusion

The new layer gives exact support-unrestricted braid compression, shared cyclic-prefix arithmetic, finite-target trie sharing, and independently replayable transformations with explicit complexity bounds. The infinite rectangle family proves that target radius matters even after all supports and all literal cyclic cuts have been allowed. The geodesic family proves that original-braid equality alone cannot universally produce short unknot words. Both negative results sharpen the design requirements instead of being hidden by successful examples.

The constructive positive result is an uncapped quasi-polynomial recognizer for the stated small-kernel and conjugated-inflation classes. The practical result is an opt-in, tested pre-processor with a cheap normal-form shortcut and measured shared-algebra savings. The general quasi-polynomial problem still requires an additional structural theorem or a different compressed representation with a complete cost proof.

A Reproduction and artifact map

The package is standard-library Python 3.10+ and was tested under 3.13.5. No network access or third-party Python package is needed for its tests. From the unpacked package root:

```
python -m unittest discover -s tests -v
python scripts/benchmark.py --rounds 7
python scripts/radius_experiment.py
python -m cyclic_garside compress certificates/rectangle_m2.json \
    --radius 2 -o result.json
python -m cyclic_garside verify certificates/rectangle_m2.json result.json
cd docs && sh build.sh
```

The Python API separates a rule-set optimum from a practical portfolio:

```
from cyclic_garside import compress, compress_radius, preprocess
from cyclic_garside import verify, verify_radius

# Optimal support-unrestricted radius-one cyclic interval plan.
result = compress(4, word)
verify(4, word, result["certificate"])

# Optimal radius-two plan; dictionary allocation is explicitly capped.
result = compress_radius(4, word, radius=2, max_targets=100000)
verify_radius(4, word, result["certificate"])

# Cheap whole-word candidate first, then a cyclic kernel if necessary.
# This output is not labeled as a radius-specific optimum.
result = preprocess(4, word, radius=2)
```

The artifact directories contain source code, independent oracles and tests, raw benchmark JSON, example certificates, this article and build sources, an exact source manifest, an integration adapter and an unexecuted upstream smoke script. The JSON schema is intentionally small and versioned; the verifier is the authoritative executable check of its local moves. The SHA-256 manifest covers the released artifact bytes, not a cryptographic endorsement of their mathematical claims.

B Audit questions for a reviewer

A review can proceed without trusting the timing conclusions. First check the two permutation conventions in Lemma 3.1; reversing starting and finishing sets is a plausible implementation error. Next check the negative-letter complement and the movement of Δ^{-1} to the left. Then verify that both certificate implementations reconstruct their actual source and never accept permutation equality as braid equality.

For the optimizer, the critical identities are $P_i = P_j v^{-1}$ and the query-before-insertion recurrence. For the cost proof, inspect negative updates, list deletions, target trie size, proof allocation, and the distinction between word and bit operations. For the separation theorem, verify the induced path and the labeled-pair crossing calculation across the unique b . For the recognition theorem, check that strand count is removed using the one-cycle permutation and that the fallback uses an actual exact homology calculation rather than an assumed short certificate.

Independent finite checks have deliberately been included for these seams. No test outcome should be used to replace a missing all-input proof, and no source-reported upstream test count should be mistaken for an executed integration test.

References

- [1] P. Dehornoy, *Efficient solutions to the braid isotopy problem*, arXiv:math/0703666 (2007), especially the simple-braid model and greedy normal form in Section 1. <https://arxiv.org/abs/math/0703666>.
- [2] P. B. Kronheimer and T. S. Mrowka, *Khovanov homology is an unknot-detector*, Publications Mathématiques de l’IHÉS **113** (2011), 97–208; arXiv:1005.4346. <https://arxiv.org/abs/1005.4346>.
- [3] M. Lackenby, *Incompressible surfaces, hierarchies and unknot recognition*, arXiv:2607.23350v1 (2026), Section 9. <https://arxiv.org/abs/2607.23350>.
- [4] V. Reshetnikov, *ProveIt: Topology/UnknotRecognition, README*, inspected blob 46709747407df3853298bf42ec1566c6ef836f3a, October 2026. Exact source paths and snapshot metadata are supplied in `source_manifest.json`. <https://github.com/VladimirReshetnikov/ProveIt/tree/main/Topology/UnknotRecognition>.
- [5] ProveIt research program, *Verified rank-two braid preprocessing, synthesis/ranktwo.tex*, inspected blob bd2943b64959697790fc71974a5442250710641d. <https://github.com/VladimirReshetnikov/ProveIt/blob/main/Topology/UnknotRecognition/synthesis/ranktwo.tex>.
- [6] ProveIt research program, *Checked elementary Markov descent, fast/fastunknot/braid_reduction.py*, inspected at commit a70135c01af3d106cec88045fa6ba8dd7d2ee0f1. https://github.com/VladimirReshetnikov/ProveIt/blob/main/Topology/UnknotRecognition/fast/fastunknot/braid_reduction.py.
- [7] OpenAI, *Positive braid derivations restrict to a full subdiagram*, lean/OAI/Topology/ArtinGroups/ParabolicIntersections/BraidRestriction.lean, commit adc7f1241b42e322a6451854ab7e4b4c146bf78a. <https://github.com/openai/math>. Inspected as methodological context; no additional result from this file is assumed in the proofs above.